

FULL STACK DEVELOPMENT

FASE 2 | AULA 02 -

INTRODUÇÃO AO EXPRESS.JS

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON	4
SAIBA MAIS	5
O QUE VOCÊ VIU NESTA AULA?	12
REFERÊNCIAS	13

EMANIP

O QUE VEM POR AÍ?

Nesta aula, exploraremos o uso do Express.js, um framework web minimalista e flexível para Node.js, que oferece um conjunto robusto de funcionalidades para aplicativos web e móveis. Vamos entender a importância de utilizar o Express.js para criar APIs e servidores web de maneira eficiente e organizada.



HANDS ON

Nesta aula prática, veremos como podemos trabalhar com o Express.js para criarmos as nossas aplicações Node.js.

EXEMPLO

SAIBA MAIS

O Express.js é um framework web minimalista e flexível para Node.js, que oferece um conjunto robusto de funcionalidades para criar aplicações web e APIs.

Ele é amplamente utilizado devido à sua simplicidade e eficiência, abstraindo a complexidade do Node.js puro ao fornecer uma API clara e intuitiva para criar servidores web.

Com Express.js, é possível definir rotas, gerenciar requisições e respostas, implementar middlewares e muito mais, de maneira organizada e escalável.

A seguir, veremos algumas das características que facilitam o desenvolvimento de nossas aplicações utilizando este framework:

Sistema de rotas completo

O Node.js, especialmente quando utilizado com frameworks como Express, permite a criação de sistemas de rotas robustos. Um sistema de rotas define como a aplicação responde a diferentes solicitações do cliente. Exemplo:

```
const express = require('express');
const app = express();

app.get('/', (req, res) => {
  res.send('Home Page');
});

app.get('/about', (req, res) => {
  res.send('About Page');
});

app.listen(3000, () => {
  console.log('Server is running on port 3000');
```

```
});
```

Tratamento de exceções

O tratamento de exceções é crucial para garantir que a aplicação continue funcionando de forma estável, mesmo quando ocorrem erros inesperados.

Exemplo:

```
app.use((err, req, res, next) => {  
  console.error(err.stack);  
  res.status(500).send('Something broke!');  
});
```

Integração de sistemas de templates

O Node.js permite a integração de vários sistemas de templates, facilitando a criação de páginas web dinâmicas. Exemplo:

```
const express = require('express');  
const app = express();  
const ejs = require('ejs');  
  
app.set('view engine', 'ejs');  
  
app.get('/', (req, res) => {  
  res.render('index', { title: 'Home Page' });  
});  
  
app.listen(3000, () => {  
  console.log('Server is running on port 3000');  
});
```

Criação de Middlewares

Middlewares são funções que têm acesso ao objeto de solicitação (req), ao objeto de resposta (res) e à próxima função de middleware no ciclo de solicitação/resposta da aplicação. Exemplo:

```
const express = require('express');
const bodyParser = require('body-parser');

const app = express();
const PORT = 3000;

// Middleware logger
const logger = (req, res, next) => {
  console.log(`${req.method} ${req.url}`);
  next();
};

// Middleware logger
app.use(logger);
app.use(bodyParser.json());

// Rotas
app.get('/', (req, res) => {
  res.send('Home Page');
});

app.get('/about', (req, res) => {
  res.send('About Page');
});

app.post('/users', (req, res) => {
  res.status(201).send('User Created');
});
```

```
app.listen(PORT, () => {  
  console.log(`Server is running on port ${PORT}`);  
});
```

Com esta implementação, cada request que for realizado nesta aplicação, será logado o método e a action. Exemplo:

```
❌ (base) thiagoadriano@Thiagos-MBP aula2 % node logger.js  
Server is running on port 3000  
^C  
○ (base) thiagoadriano@Thiagos-MBP aula2 % node logger.js  
Server is running on port 3000  
GET /  
GET /about  
POST /users  
□
```

Figura 1 - Exemplo de retorno de middler
Fonte: elaborado pelo autor (2024)

Autenticação e autorização

A autenticação e autorização são componentes essenciais para a segurança de uma aplicação. Utilizando o express.js conseguimos configurar este fluxo com poucas linhas de código:

```
const express = require('express');  
const bodyParser = require('body-parser');  
const passport = require('passport');  
const LocalStrategy = require('passport-local').Strategy;  
const session = require('express-session');  
  
const app = express();  
const PORT = 3000;  
  
// Middleware de sessão
```



```
app.use(session({
  secret: 'secretKey',
  resave: false,
  saveUninitialized: true
}));

// Middleware do Passport
app.use(passport.initialize());
app.use(passport.session());

app.use(bodyParser.json());
app.use(bodyParser.urlencoded({ extended: false }));

// Configuração do Passport LocalStrategy
passport.use(new LocalStrategy(
  (username, password, done) => {
    // Lógica de autenticação
    if (username === admin && password === 1010) {
      return done(null, { id: 1, name: 'Thiago S Adriano' });
    } else {
      return done(null, false, { message: 'Invalid credentials' });
    }
  }
));

// Serialização do usuário
passport.serializeUser((user, done) => {
  done(null, user.id);
});

// Desserialização do usuário
passport.deserializeUser((id, done) => {
  // Aqui você deve buscar o usuário no banco de dados
});
```

```
// Aqui estamos simulando com um usuário fixo
const user = { id: 1, name: ' Thiago S Adriano' };
done(null, user);
});

// Middleware de log
const logger = (req, res, next) => {
  console.log(`${req.method} ${req.url}`);
  next();
};

app.use(logger);

// Rota para login
app.post('/login', passport.authenticate('local', {
  successRedirect: '/dashboard',
  failureRedirect: '/login',
  failureFlash: true
}));

// Rota protegida (dashboard)
app.get('/dashboard', (req, res) => {
  if (req.isAuthenticated()) {
    res.send('Welcome to your dashboard!');
  } else {
    res.redirect('/login');
  }
});

// Rota de login (simplesmente uma página de login para o exemplo)
app.get('/login', (req, res) => {
  res.send('<form action="/login" method="post">\n
    <div>\n
```

```
    <label>Username:</label>\n    <input type="text" name="username"/>\n  </div>\n  <div>\n    <label>Password:</label>\n    <input type="password" name="password"/>\n  </div>\n  <div>\n    <input type="submit" value="Log In"/>\n  </div>\n</form>');\n});\n\n// Tratamento de exceções\napp.use((err, req, res, next) => {\n  console.error(err.stack);\n  res.status(500).send('Something broke!');\n});\n\napp.listen(PORT, () => {\n  console.log(`Server is running on port ${PORT}`);\n});
```

O QUE VOCÊ VIU NESTA AULA?

Nesta aula, exploramos o uso do Express.js no desenvolvimento de aplicações web e APIs com Node.js. Aqui estão os principais pontos abordados:

- **Definição do Express.js:** entendemos a importância e os benefícios de usar o Express.js para desenvolver servidores web com Node.js.
- **Exemplo de Aplicação Básica:** configuramos e criamos uma aplicação básica com Express.js.
- **Implementação de Rotas:** criamos rotas para manipular diferentes requisições HTTP.
- **Uso de Middleware:** implementamos middlewares para processar requisições e respostas de maneira organizada.

REFERÊNCIAS

PAULA, A. O que é o Express.js?. **treinaweb**. 2021. Disponível em: https://www.treinaweb.com.br/blog/o-que-e-o-express-js#google_vignette. Acesso em: 19 ago. 2024.

O Que é Express.js? Tudo o Que Você Precisa Saber. **kinsta**. 2022. Disponível em: <https://kinsta.com/pt/base-de-conhecimento/o-que-e-express-js/>. Acesso em: 19 ago. 2024.

Express.js Tutorial. **geeksforgeeks**. 2024. Disponível em: <https://www.geeksforgeeks.org/express-js/>. Acesso em: 19 ago. 2024.

PALAVRAS-CHAVE

Palavras-chave: Express, Middler, Node.js.

EXEMPLO